

Glosario - Semana 05

A

Alembic

Herramienta de migración de bases de datos para SQLAlchemy. Permite versionar cambios en el esquema de la base de datos.

B

Base (DeclarativeBase)

Clase base de SQLAlchemy de la que heredan todos los modelos. Define el registro de metadatos para la creación de tablas.

```
from sqlalchemy.orm import DeclarativeBase

class Base(DeclarativeBase):
    pass
```

C

Commit

Operación que guarda permanentemente los cambios pendientes en la base de datos.

```
session.add(user)
session.commit() # Guarda el usuario en la DB
```

Connection Pool

Conjunto de conexiones de base de datos reutilizables que mejora el rendimiento evitando crear conexiones nuevas constantemente.

CRUD

Acrónimo de Create, Read, Update, Delete. Las cuatro operaciones básicas de persistencia de datos.

D

DeclarativeBase

Nueva clase base en SQLAlchemy 2.0 para definir modelos usando el estilo declarativo con type hints.

Dependency Injection

Patrón de diseño donde las dependencias (como la sesión de DB) se "inyectan" en las funciones que las necesitan.

```
def get_user(db: Session = Depends(get_db)):
    # db se inyecta automáticamente
```

E

Engine

Objeto central de SQLAlchemy que gestiona la conexión a la base de datos y el pool de conexiones.

```
engine = create_engine("sqlite:///./app.db")
```

F

Foreign Key

Restricción que crea una relación entre dos tablas, referenciando la clave primaria de otra tabla.

```
author_id: Mapped[int] = mapped_column(ForeignKey("authors.id"))
```

Flush

Operación que sincroniza los cambios en memoria con la base de datos, pero sin hacer commit (sin confirmar permanentemente).

I

Index

Estructura de datos que mejora la velocidad de las consultas en una columna específica.

```
email: Mapped[str] = mapped_column(index=True)
```

M

Mapped

Tipo genérico de SQLAlchemy 2.0 que indica el tipo Python de una columna.

```
name: Mapped[str] # Columna de tipo string
age: Mapped[int | None] # Columna nullable
```

mapped_column

Función que configura una columna de SQLAlchemy con opciones adicionales.

```
id: Mapped[int] = mapped_column(primary_key=True)
```

Metadata

Colección de objetos Table que representa la estructura de la base de datos.

```
Base.metadata.create_all(bind=engine)
```

N

Nullable

Columna que permite valores NULL. En SQLAlchemy 2.0 se indica con `T | None` .

```
phone: Mapped[str | None] # Puede ser NULL
```

O

ORM (Object-Relational Mapping)

Técnica que permite trabajar con bases de datos usando objetos Python en lugar de SQL directo.

P

Primary Key

Columna que identifica únicamente cada fila de una tabla.

```
id: Mapped[int] = mapped_column(primary_key=True)
```

Q

Query

Consulta a la base de datos. En SQLAlchemy 2.0 se usa `select()` en lugar del antiguo `query()` .

```
stmt = select(User).where(User.age > 18)
result = session.execute(stmt)
```

R

Refresh

Operación que recarga un objeto desde la base de datos para obtener valores generados (como IDs autoincrementales).

```
session.refresh(user) # Actualiza user.id después del commit
```

Rollback

Operación que deshace todos los cambios pendientes desde el último commit.

```
try:
    session.add(user)
    session.commit()
except:
    session.rollback()
```

S

Scalar

Método que retorna un solo valor de una consulta (en lugar de una fila completa).

```
user = session.execute(stmt).scalar_one_or_none()
```

Session

Objeto que gestiona la conversación con la base de datos. Maneja transacciones y el ciclo de vida de los objetos.

sessionmaker

Factory que crea sesiones con configuración predefinida.

```
SessionLocal = sessionmaker(bind=engine, autocommit=False)
```

T

Transaction

Conjunto de operaciones que se ejecutan como una unidad atómica. O todas tienen éxito, o ninguna.

U

Unique Constraint

Restricción que garantiza que no haya valores duplicados en una columna.

```
email: Mapped[str] = mapped_column(unique=True)
```

Unit of Work

Patrón de diseño implementado por Session que agrupa operaciones de base de datos.

Y

Yield

Palabra clave de Python usada en `get_db()` para crear un generador que provee la sesión y garantiza su cierre.

```
def get_db():
    db = SessionLocal()
    try:
        yield db # Provee la sesión
    finally:
        db.close() # Siempre se ejecuta
```

[← Volver a Semana 05](#)